



LABORATOIRE L3i

RAPPORT DE STAGE

AMINE OUBAIDI - MASTER INFORMATIQUE

Développement d'un Système de Suivi Multi-Souris pour l'analyse comportementale

Entreprise d'accueil : Laboratoire L3i, Av. Michel Crépeau, 17042 La Rochelle
Téléphone : 05 46 45 82 62

Tuteur : FRANCO Patrick

Maître de stage : Jean-Christophe BURIE

2024-2025

Remerciements

Je tiens à exprimer ma profonde gratitude envers l'ensemble des personnes qui m'ont accompagné et soutenu tout au long de ce stage au sein du Laboratoire L3i de l'Université de La Rochelle.

Mes premiers remerciements vont à mon encadrant de stage, pour son accompagnement constant, sa disponibilité et ses précieux conseils qui ont été déterminants dans la réussite de ce projet. Sa vision experte et son approche pédagogique m'ont permis de progresser significativement dans ma compréhension des problématiques de vision par ordinateur et d'apprentissage profond.

Je souhaite également remercier chaleureusement l'ensemble de l'équipe du Laboratoire L3i pour leur accueil et leur soutien. Le laboratoire a mis à ma disposition des ressources techniques essentielles, notamment un poste de travail performant et l'accès au serveur de calcul L3iCalculMaster, qui ont été cruciaux pour l'entraînement des modèles de deep learning et le traitement des données massives générées par ce projet. Ces ressources computationnelles ont permis d'accélérer significativement le développement et l'optimisation des algorithmes de détection et de suivi.

L'environnement stimulant du L3i, les échanges enrichissants avec les chercheurs et doctorants, ainsi que l'excellence des conditions de travail ont grandement contribué à la qualité des résultats obtenus. Cette expérience m'a non seulement permis d'acquérir des compétences techniques pointues, mais aussi de découvrir le monde de la recherche académique dans des conditions optimales.

Je garde un souvenir particulièrement positif de cette collaboration et de l'esprit d'entraide qui règne au sein du laboratoire, des qualités qui ont été déterminantes dans l'aboutissement de ce projet ambitieux.

Résumé

Ce stage a porté sur le développement d'un système innovant de détection et de suivi de souris en laboratoire, combinant vision par ordinateur et deep learning. Le projet a abouti à la création d'une solution complète permettant l'analyse automatisée du comportement animal.

Le système développé repose sur une architecture sophistiquée intégrant un modèle Keypoint R-CNN modifié, capable de détecter avec précision sept points anatomiques clés sur chaque souris. Cette approche permet non seulement le suivi simultané de plusieurs souris, mais aussi l'analyse fine de leurs postures et interactions. Les innovations majeures incluent l'implémentation d'un système de correction anatomique robuste, un module de détection d'interactions sociales, et une interface utilisateur intuitive facilitant l'analyse des données comportementales.

Les résultats obtenus démontrent une amélioration significative par rapport aux solutions existantes, notamment en termes de précision de détection ($>70\%$) et de capacité à gérer les situations complexes comme les occlusions partielles et les interactions entre souris. Le système permet également une analyse en temps réel, avec un traitement optimisé des flux vidéo et une génération automatique de métriques comportementales.

Ce projet a nécessité la mise en œuvre de compétences variées, allant de la programmation Python avancée à l'utilisation de frameworks de deep learning, le prétraitement, en passant par le développement d'interfaces graphiques et l'analyse statistique. Les défis techniques rencontrés ont été surmontés grâce à une approche méthodique et itérative, permettant des améliorations continues du système.

L'impact de ce travail s'étend au-delà du cadre technique, contribuant significativement à la recherche en neurologie en relation avec l'analyse du comportement d'animal en laboratoire sous influence de différents molécules dans notre cas les souris sont sous influence de TMAO triméthylamine N-oxide qui est suspecté de causer des troubles psychiatriques et neurodégénératifs.

Abstract

This internship focused on developing an innovative system for laboratory mouse detection and tracking, combining computer vision and deep learning technologies. The project resulted in the creation of a comprehensive solution for automated animal behavior analysis.

The developed system is built on a sophisticated architecture incorporating a modified Keypoint R-CNN model, capable of accurately detecting seven key anatomical points on each mouse. This approach enables not only simultaneous tracking of multiple mice but also detailed analysis of their postures and interactions. Major innovations include the implementation of a robust anatomical correction system, a social interaction detection module, and an intuitive user interface facilitating behavioral data analysis.

The results demonstrate significant improvements over existing solutions, particularly in terms of detection accuracy ($>70\%$) and the ability to handle complex situations such as partial occlusions and mouse interactions. The system also enables real-time analysis, with optimized video stream processing and automatic generation of behavioral metrics.

This project required the implementation of diverse skills, ranging from advanced Python programming to the use of deep learning frameworks, preprocessing techniques, graphical interface development, and statistical analysis. Technical challenges were overcome through a methodical and iterative approach, enabling continuous system improvements.

The impact of this work extends beyond the technical framework, contributing significantly to neurological research related to laboratory animal behavior analysis under the influence of various molecules. Specifically, in our case, the mice are under the influence of TMAO (trimethylamine N-oxide), a compound suspected of causing psychiatric and neurodegenerative disorders.

Table des matières

1	Présentation de l'unité d'accueil : le L3i	3
2	Contexte et objectifs du stage	4
3	Définitions et état de l'art	6
4	Approche proposée	8
4.1	Architecture du Modèle	9
4.2	Prédicteur des points caractéristiques personnalisés	9
4.3	Justification des Choix Architecturaux	10
4.3.1	Architecture de la structure de base	10
4.3.2	Tête de Prédiction Personnalisée	10
4.3.3	Configuration des ancres	11
4.3.4	Stratégie d'apprentissage par transfert (Transfer Learning)	11
4.3.5	Paramètres d'Entraînement	12
4.4	Architecture du Pipeline de Réglage Fin (fine tuning)	13
5	Résultats et Améliorations	13
5.1	Gains de Performance	13
5.2	Analyse des Améliorations	13
6	Pipeline de Données	14
6.1	Chargement et Prétraitement des Données	14
6.2	Jeu de données	15
7	Implémentation du logiciel de Suivi	16
7.1	Architecture Logicielle	16
7.2	Interface Graphique (GUI)	17
7.2.1	Panneau de Contrôle Principal	17
7.2.2	Panneau des Paramètres Biomécaniques	17
7.2.3	Panneau d'Analyse Avancée	18
8	Détail de la Pipeline	19
8.1	Modèle de détection	19
8.2	Prétraitement des images	19
8.3	Post-traitement des détections	20
8.3.1	Non-Maximum Suppression (NMS)	20
8.3.2	Filtrage et assignation par souris	20
8.3.3	Gestion des cas particuliers	21
8.4	Assignation Stable des IDs	21
8.4.1	Identifiants Globaux	21
8.4.2	Algorithme d'Assignation	22
8.5	Gestion des Interactions et Enchevêtrements	23
8.5.1	Détection des Rapprochements	23
8.5.2	Séparation des Détections Fusionnées	23
8.5.3	Gestion des Occlusions	23

8.5.4	Métriques de Qualité	24
9	Pipeline de Traitement	25
9.1	Détection Brute	25
9.2	Filtrage NMS et Seuillage	26
9.3	Sélection des N "Meilleures" Souris	26
9.4	Correction Anatomique	27
9.5	Gestion de la proximité des souris	27
9.6	Fichier de sortie	28
	Annexes	33
A	Bibliographie	33
B	Algorithme d'entraînement	34
C	Chargement des données	35
D	Filtre de Kalman pour le Suivi	36

1 Présentation de l'unité d'accueil : le L3i

Le L3i (Laboratoire Informatique, Image et Interaction) est une unité de recherche rattachée à La Rochelle Université. Créé en 1993, en même temps que l'établissement, le laboratoire a obtenu dès 1997 le label d'équipe d'accueil (EA 2118) du Ministère de l'Enseignement Supérieur et de la Recherche. Depuis, il n'a cessé de croître pour atteindre aujourd'hui un effectif d'environ une centaine de membres, répartis entre enseignants-chercheurs, doctorants, post-doctorants et ingénieurs. Il constitue l'une des principales structures de recherche du domaine STIC (Sciences et Technologies de l'Information et de la Communication) à La Rochelle Université. Le L3i se distingue par son positionnement à la croisée de plusieurs disciplines liées à l'informatique, à l'image, à l'interaction et à la gestion intelligente de contenus numériques. Il regroupe la grande majorité des enseignants-chercheurs en informatique et en génie informatique de l'université, issus notamment des départements Informatique et Réseaux et Télécoms (RT) de l'IUT ainsi que de l'institut LUDI.

Dans un contexte d'évolution rapide et massive des technologies numériques — marqué notamment par l'essor de l'intelligence artificielle, l'omniprésence des objets connectés et la dématérialisation des services, le L3i conduit des travaux de recherche appliquée centrés sur les contenus numériques et les interactions homme-machine. Ces travaux abordent des problématiques concrètes et actuelles : traitement automatique des données, extraction de connaissances, accessibilité et personnalisation de l'information, interactions enrichies avec des environnements numériques complexes, respect de la vie privée, optimisation énergétique des systèmes, etc.

Organisation scientifique

Le L3i articule sa recherche autour d'un projet scientifique fédérateur : la gestion interactive et intelligente des contenus numériques. Cette orientation s'appuie sur une structuration en trois équipes complémentaires et interconnectées :

- **IC – Images et Contenus** : spécialisée dans le traitement bas niveau des contenus faiblement structurés (images, vidéos, textes, documents), cette équipe travaille sur l'extraction de caractéristiques, la recherche d'information et l'indexation.
- **eAdapt – Dynamique des systèmes et adaptativité** : cette équipe s'intéresse aux systèmes logiciels dynamiques et adaptatifs, notamment dans les interactions homme-système et les systèmes distribués ou en réseau.
- **MC – Modèles et Connaissances** : elle développe des approches formelles pour la modélisation de connaissances, l'analyse de données complexes (spatio-temporelles, réseaux de concepts), l'intégration de données et les raisonnements associés.

En complément, le laboratoire structure son activité autour de trois **axes transverses**, permettant de favoriser les collaborations interdisciplinaires et les interactions avec le monde socio-économique :

- **Supervision environnementale et littorale** : développement de plateformes de monitoring du littoral (capteurs, drones, IA), en lien avec la modélisation de données environnementales.

- **Humanités numériques** : gestion de contenus numériques complexes issus de la culture ou de l'éducation, contextualisation des traitements, IA sémantique et interactions riches avec l'utilisateur.
- **Numérique et société** : réflexion sur les impacts éthiques, sociaux et environnementaux du numérique, autour de thématiques comme la confiance numérique, l'économie circulaire ou la réduction de l'empreinte carbone des systèmes.

Organigramme

Le laboratoire est composée d'environ une centaine de personnes. Il se décompose avec un tiers de permanents, un tiers de post-doctorants et un tiers de doctorant. L'organigramme du laboratoire est représenté sur la figure 1.

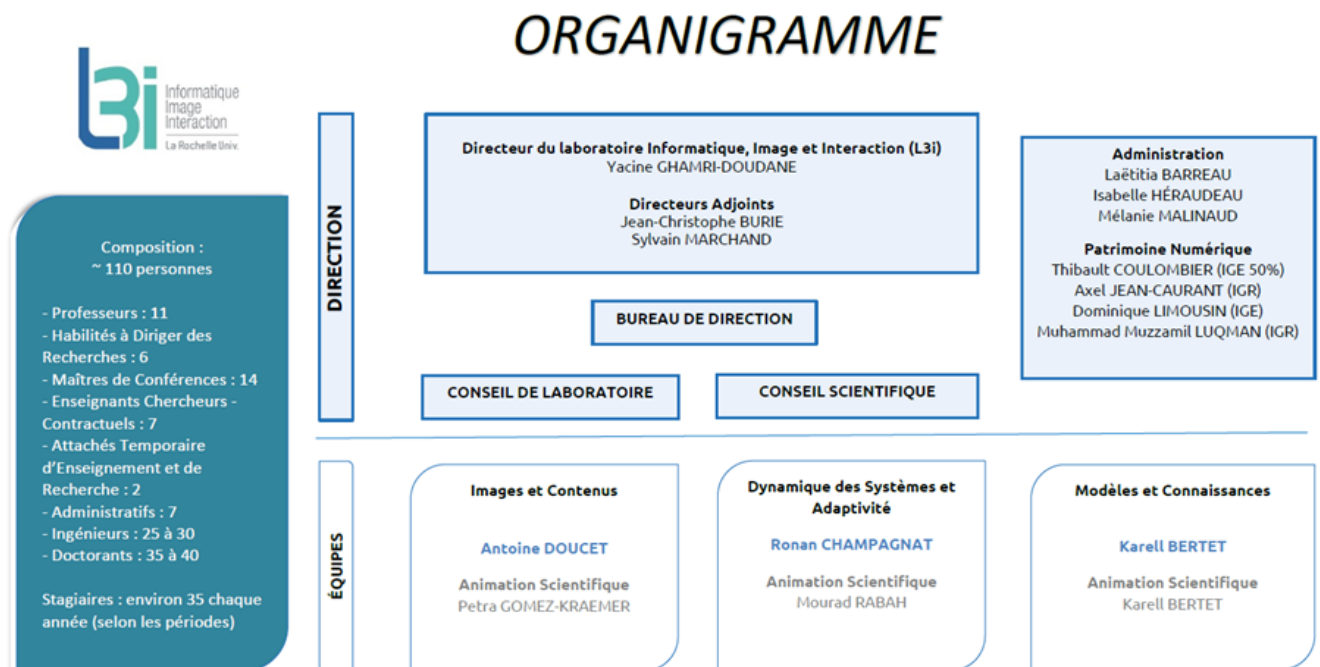


FIGURE 1 – Organigramme

2 Contexte et objectifs du stage

Contexte

Dans le cadre de mon stage de Master au sein de l'équipe « Images et Contenus » du laboratoire L3i, j'ai été amené à travailler sur l'analyse comportementale de souris de laboratoire à partir de vidéos. L'objectif global de ce type de travaux est de permettre une observation automatisée des comportements animaux.

L'approche envisagée initialement reposait sur l'utilisation d'un outil nommé *Live Mouse Tracker* (LMT), une solution conçue pour le suivi en temps réel de souris dans des environnements expérimentaux à l'aide de caméras spécifiques.

Contexte Scientifique

Ce stage s'inscrit dans le cadre du projet collaboratif **TMAO-Behavior**, mené en partenariat avec l'Université Catholique de Valence, dont l'objectif est d'étudier les effets biologiques et fonctionnels de la TMAO (triméthylamine-N-oxyde) sur le cerveau et le comportement animal.

La TMAO est une molécule produite par le métabolisme, à partir de composés abondant dans notre alimentation, via l'action du microbiote intestinal. Des études préliminaires suggèrent qu'à fortes doses, cette molécule pourrait altérer significativement les comportements et les interactions sociales chez les rongeurs.

L'analyse quantitative de ces comportements vise à :

- Établir une corrélation entre dosage de TMAO et modifications comportementales
- Compter les interactions sociales

Objectifs initiaux et évolution de la mission

L'objectif initial de ma mission était donc de tester et d'évaluer le logiciel *Live Mouse Tracker* (LMT) sur un ensemble de vidéos expérimentales fournies par la biologiste de l'Université Catholique de Valence. Cependant rapidement une limite majeure a été identifiée : L'utilisation des fonctionnalités de suivi les plus récentes de l'outil LMT n'était possible qu'à partir de flux vidéo "direct" issus de caméras de type "Kinect". Il n'était pas possible d'analyser les vidéos brutes préenregistrées qui nous avaient été transmises.

En étudiant le logiciel LMT, nous avons découvert qu'il pouvait utiliser des vidéos enregistrées. Cependant le suivi était possible en utilisant une ancienne méthode datant de 2012. Le point faible de la méthode est qu'il nécessite une initialisation manuelle en indiquant sur chaque souris le point correspondant à la tête et celui correspondant à la queue. De plus, les tests réalisés ont montré que la méthode n'était pas fiable. En effet, nous avons constaté deux problèmes :

- le suivi n'était pas stable. Au cours de la séquence, l'algorithme perdait souvent les souris.
- lorsque les souris se touchaient, l'algorithme inversait souvent leurs identifiants ne permettant pas d'analyser de manière fiable leurs trajectoires

Face à la contrainte technique nécessitant une caméra spécifique et le manque de fiabilité du seul algorithme disponible, il a été décidé de réorienter le stage vers la conception et le développement d'une solution alternative adaptée à notre cas d'usage : le suivi automatisé de souris sur des vidéos existantes.

Redéfinition des objectifs

J'ai ainsi proposé la mise en place d'un pipeline de traitement vidéo dédié au suivi d'animaux, avec pour objectifs principaux :

- détecter et suivre automatiquement les souris dans les vidéos fournies ;
- détecter 7 points anatomiques caractéristiques pour chaque souris (le nez, le cou, l'oreille droite, l'oreille gauche, le flan droit, le flan gauche et la base de la queue) afin de déterminer les interactions sociales (contact nez à nez, nez à queue, etc).
- d'extraire des données exploitables pour l'analyse comportementale ;
- de proposer une solution générique, non dépendante de matériel spécifique.

Ce nouveau cadre de travail a donné lieu à une exploration des techniques de vision par ordinateur, d'apprentissage automatique et d'outils open-source adaptés au traitement de vidéos animales. Les choix méthodologiques, les résultats obtenus et les difficultés rencontrées seront détaillés dans les sections suivantes.

3 Définitions et état de l'art

Concepts fondamentaux

Réseau de neurones convolutifs (CNN) : Ce type de réseau de neurones artificiels est spécialement conçu pour traiter des données structurées en grille, comme les images. Les CNN utilisent des opérations de convolution pour extraire automatiquement des caractéristiques visuelles à différents niveaux d'abstraction, de simples contours à des formes complexes.

Apprentissage profond (Deep Learning) : Ce terme fait référence à Une branche de l'intelligence artificielle qui utilise des réseaux de neurones avec plusieurs couches cachées pour apprendre des représentations hiérarchiques des données. Contrairement aux méthodes traditionnelles qui nécessitent une extraction manuelle des caractéristiques, l'apprentissage profond permet au système d'apprendre automatiquement les caractéristiques pertinentes à partir des données qui sont présentées au réseau.

Apprentissage par transfert (Transfer learning) : Cette technique d'apprentissage automatique consiste à réutiliser un modèle pré-entraîné sur une tâche différente mais similaire. Cette approche permet de bénéficier des connaissances acquises sur de grandes bases de données et d'adapter le modèle à notre problème spécifique avec moins de données d'entraînement.

Architectures et modèles

Il existe dans la littérature scientifique de nombreuses architectures (CNN, RNN, etc). Ces architectures ont été développés afin de répondre à différentes contraintes comme la détection temps-réel (YOLO, Fast R-CNN) ou pour des applications spécifiques comme le UNET appliqué dans un premier temps aux image médicales.

Dans ce travail, nous nous sommes inspirés des travaux de ??? qui utilise les éléments suivants :

ResNet50 : c'est un réseau de neurones convolutifs profond de 50 couches, introduit par Microsoft Research en 2015. Sa particularité réside dans l'utilisation de connexions résiduelles (skip connections) qui permettent de contourner le problème du gradient qui disparaît lors de l'entraînement de réseaux très profonds.

Feature Pyramid Network (FPN) : cette architecture permet d'extraire des caractéristiques à plusieurs échelles simultanément. Elle combine des caractéristiques de bas niveau (détails fins) et de haut niveau (formes globales) pour améliorer la détection d'objets de différentes tailles.

Keypoint R-CNN : C'est une architecture de détection d'objets qui combine la détection de boîtes englobantes avec la localisation précise de points clés anatomiques. Cette approche permet de détecter non seulement la présence d'un objet, mais aussi sa pose détaillée.

Algorithmes de suivi

Les algorithmes de suivi permettent de "suivre", dans une séquence d'images ou une vidéos les objets d'intérêts. Dans la littérature, on trouve différentes approches qui peuvent être complémentaires selon le contexte d'application. Dans nos travaux, nous avons étudié plus précisément les trois approches suivantes :

Filtre de Kalman : il s'agit d'un algorithme mathématique qui permet d'estimer l'état d'un système dynamique à partir de mesures bruitées et incomplètes. Dans notre contexte, il est utilisé pour prédire la position future des souris et lisser les trajectoires en combinant les prédictions avec les nouvelles observations.

Algorithme hongrois : C'est un algorithme d'optimisation qui résout le problème d'assignation optimale. Dans notre système, il est utilisé pour associer les détections de souris d'une frame à l'autre en minimisant la somme des distances entre les positions prédites et détectées.

Non-Maximum Suppression (NMS) : Cette technique de post-traitement s'applique pour éliminer les détections redondantes en conservant seulement les détections avec les scores de confiance les plus élevés dans chaque zone de chevauchement.

Concepts biomécaniques

Dans ce travail, les points anatomiques devaient respecter une cohérence spatiale et temporelle. Il était donc impératif d'imposer certaines règles. Nous avons donc considéré dans notre travail, les points ci-dessous.

Validation biomécanique : Ce processus vérifie la plausibilité anatomique des poses détectées en appliquant des contraintes basées sur la physiologie de la souris, comme les proportions corporelles, les angles articulaires et la symétrie.

Interpolation temporelle : Cette technique estime les positions manquantes en utilisant les informations des frames précédentes et suivantes. Cette méthode est particulièrement utile lors d'occlusions temporaires ou de détections manquantes.

Contraintes biomécaniques : Issue de l'historique, ces contraintes viennent maintenir la cohérence des points anatomiques ; ces contraintes sont représentées par les distances et angles entre les points.

Partitionnement (Clustering) : Cet ensemble de techniques regroupent les données ayant des propriétés similaires. Dans notre cas, l'algorithme K-means est utilisé pour séparer les points caractéristiques des deux souris qui ont été détectées comme une seule entité.

4 Approche proposée

Pour réaliser un système robuste de suivi multi-souris, plusieurs méthodes et techniques d'apprentissage profond ont été mises en œuvre. L'architecture KeypointRCNN, basée sur une structure ResNet50-FPN, a été utilisée pour l'extraction de caractéristiques visuelles hiérarchiques à partir des images. Cette architecture a été enrichie par une tête de prédiction personnalisée intégrant des couches de convolution adaptatives et des mécanismes de régularisation spécifiques.

La détection et le suivi des points anatomiques clés reposent sur plusieurs composants essentiels :

- **Réseau pyramidal de caractéristiques (FPN)** : Ce réseau permet l'analyse multi-échelle des caractéristiques visuelles, crucial pour gérer les variations de taille des souris dans l'image
- **Réseau de proposition de région (RPN)** : Ce réseau a pour objectif l'identification des régions d'intérêt. Il est optimisé avec des ancres (anchors) spécifiques ((32,), (64,), (128,), (256,), (512,)) pour la détection précise des boîtes englobantes
- **Tête de régression** : Il s'agit d'une architecture à deux branches utilisant des convolutions 3×3 suivies de convolution 1×1 pour la localisation précise des points d'intérêts.

Pour améliorer la robustesse et permettre la généralisation du modèle, plusieurs techniques de régularisation ont été implémentées :

- **Abandon spatial (valeur utilisée : 30%)** : Il réduit la co-adaptation des caractéristiques et améliore la robustesse aux variations de pose.
- **Décroissance des poids (valeur utilisée : 0.01)** : Contrôle la complexité du modèle et prévient le surapprentissage
- **Arrêt précoce** : Surveillance de la perte de validation avec un seuil de patience de 6 époques (nombre de passages qu'un jeu de données d'entraînement effectue lors de la phase d'apprentissage).

L'adaptation au domaine cible (vidéos de laboratoire) a été réalisée via une stratégie de réglage fin différenciée, avec des taux d'apprentissage distincts pour la structure de base (3×10^{-5}) et la tête de prédiction (5×10^{-4}). Cette approche permet de préserver les caractéristiques génériques de bas niveau tout en adaptant les couches supérieures aux spécificités de notre tâche.

4.1 Architecture du Modèle

L'architecture de notre modèle est implémentée en PyTorch. Il s'agit d'une bibliothèque logicielle Python open source d'apprentissage automatique qui s'appuie sur Torch, outil développée par Meta. L'architecture s'articule autour de deux composants principaux.

- La première partie est inspirée de l'article ?? qui utilise la même architecture. Dans cet article, elle est utilisée pour la détection de 17 points anatomique du corps humain. Nous l'avons adapté pour notre cas d'étude : la détection des souris.
- La seconde partie "personnalisée" est construite sur une tête de prédiction adaptée aux 7 points anatomiques des souris.

Cette architecture, représentée sur la figure 2, permet donc à la fois la détection des souris et la détection des 7 points anatomiques des souris détectées.



FIGURE 2 – Architecture du modèle de détection des points anatomiques

4.2 Prédicteur des points caractéristiques personnalisés

Le listing 1 montre l'implémentation du prédicteur de point personnalisé.

```
1 class CustomKeypointRCNNPredictor(nn.Module):
2     def __init__(self, in_channels, num_keypoints):
3         super().__init__()
4         self.head = nn.Sequential(
5             nn.Conv2d(in_channels, 512, kernel_size=3, padding=1),
6             nn.ReLU(),
7             nn.Dropout(0.3),
8             nn.Conv2d(512, num_keypoints, kernel_size=1)
9         )
10    def forward(self, x):
11        return self.head(x)
```

Listing 1 – Implémentation du Prédicteur de points caractéristiques personnalisés

Il est caractérisé par les éléments suivants :

- Une **couche d'entrée** matérialisée par une convolution 3×3 avec un paramètre préservant les dimensions spatiales (padding).
- Une **couche d'activation**
- Une **couche de régularisation** pour prévenir le surapprentissage (Dropout de 30%)
- Une **couche de sortie** matérialisé par une couche de convolution 1×1 pour la projection vers l'espace des points caractéristiques.

Composants Clés du modèle Le modèle est initialisé à l’aide de poids pré-entraînés issus du dataset COCO_V1, ce qui permet de bénéficier d’un apprentissage déjà effectué sur un large corpus d’images. Il est possible de geler la structure de base du réseau, c’est-à-dire de ne pas la modifier pendant l’entraînement, afin de faciliter le transfert d’apprentissage. Toutefois, un réglage plus fin peut également être appliqué de manière sélective à certaines couches pour optimiser les performances sur des données spécifiques.

Concernant la configuration du RPN (Region Proposal Network), les paramètres ont été adaptés aux dimensions des souris, ce qui est essentiel pour améliorer la détection des objets de petite taille. Le modèle utilise cinq tailles d’ancres différentes, allant de 32 à 512 pixels, ainsi que trois rapports d’aspect (0.5, 1.0 et 2.0) pour couvrir une diversité de formes et de proportions.

Avantages de l’Architecture L’architecture du modèle présente plusieurs avantages. Elle est d’abord très flexible grâce à sa structure modulaire, ce qui permet d’adopter différentes stratégies d’entraînement selon les besoins. Elle se montre également efficace, notamment en tirant parti de l’apprentissage transféré, qui permet de réutiliser des caractéristiques déjà apprises sur d’autres jeux de données. L’adaptabilité constitue un autre atout important : il est possible de configurer précisément les ancres et les transformations pour mieux s’ajuster aux données ciblées. Enfin, le modèle intègre des mécanismes de régularisation qui renforcent sa robustesse face aux variations des données d’entrée.

4.3 Justification des Choix Architecturaux

4.3.1 Architecture de la structure de base

Nous avons choisi une architecture basée sur le modèle ResNet50 couplé à un FPN (Feature Pyramid Network), pour des raisons à la fois techniques et adaptées à notre cas d’usage. Le FPN s’est imposé en raison de sa capacité à gérer efficacement les variations d’échelle, particulièrement importantes dans la détection de souris. Il permet d’extraire des caractéristiques à plusieurs niveaux de résolution, ce qui améliore sensiblement la détection des petits objets.

Le ResNet50, quant à lui, constitue un bon compromis entre profondeur du réseau, complexité et performance. Sa profondeur est suffisante pour capter des représentations riches tout en restant maîtrisable sur le plan computationnel. Par ailleurs, l’utilisation de connexions résiduelles contribue à atténuer le problème de la disparition du gradient lors de l’entraînement, ce qui renforce la stabilité et l’efficacité du modèle.

4.3.2 Tête de Prédiction Personnalisée

La tête de prédiction que nous avons développée, nommée CustomKeypointRCNNPredictor (voir Listing 1), répond à des besoins spécifiques liés à la localisation de points anatomiques sur les souris.

Elle débute par une couche convolutive de taille 3×3 , définie par la commande `nn.Conv2d(in_channels, 512, kernel_size=3, padding=1)`. Ce choix permet de cap-

ter efficacement le contexte local autour des points clés tout en maintenant les dimensions spatiales grâce au padding (technique de remplissage permettant de garder des images de taille constante). L'utilisation de 512 filtres assure une représentation suffisamment riche des caractéristiques extraites.

Pour renforcer la robustesse du modèle et limiter les risques de surapprentissage, nous avons intégré une régulation du modèle par une couche d'abandon (dropout) avec un taux fixé à 30%, implémentée via la commande `nn.Dropout(0.3)`. Ce taux a été déterminé de manière empirique pour optimiser la régularisation, en particulier face à la variabilité des poses.

Enfin, la projection finale est réalisée à l'aide d'une convolution 1×1 , par la commande `nn.Conv2d(512, num_keypoints, kernel_size=1)`. Cette couche permet une réduction efficace de la dimensionnalité tout en préservant les informations spatiales, avec un coût computationnel réduit.

4.3.3 Configuration des ancres

En détection d'objets, les ancres (anchor boxes) sont des boîtes rectangulaires prédéfinies, placées régulièrement sur l'image, qui servent de points de départ pour prédire la position des objets. Chaque ancre a une taille et un ratio d'aspect fixes (par exemple : petite et allongée, moyenne et carrée, grande et verticale, etc.).

La configuration des ancres joue un rôle crucial dans la qualité de la détection. Nous avons choisi des tailles d'ancres spécifiques, correspondant à ((32,), (64,), (128,), (256,), (512,)), de manière à couvrir l'ensemble des tailles de souris couramment observées. Cette progression géométrique permet une détection multi-échelle performante et s'adapte aux variations de distance entre l'objet et la caméra.

Concernant les ratios d'aspect, nous avons utilisé ((0.5, 1.0, 2.0,)) pour chaque taille d'ancre. Chaque ratio remplit un objectif précis : 0.5 permet de mieux détecter les poses allongées, 1.0 correspond aux poses standard, et 2.0 est adapté aux configurations plus verticales. Cette diversité contribue à rendre le modèle plus robuste face à la variabilité des postures.

4.3.4 Stratégie d'apprentissage par transfert (Transfer Learning)

La stratégie de *transfer learning* que nous avons adoptée repose sur un gel sélectif de la structure de base (appelé backbone en anglais). Il est possible de geler entièrement ou partiellement les couches du réseau selon les besoins. Cette flexibilité permet à la fois de préserver les caractéristiques de bas niveau, souvent universelles, et d'adapter l'apprentissage aux spécificités de notre jeu de données. Cette approche permet également de réduire le temps d'entraînement, tout en maintenant une bonne capacité d'adaptation du modèle.

Algorithme d'Entraînement avec Régularisation

L'entraînement du modèle suit une approche rigoureuse intégrant plusieurs mécanismes de régularisation pour assurer une convergence stable et éviter le surapprentissage comme le montre la figure 3. L'algorithme est détaillé en annexe B.

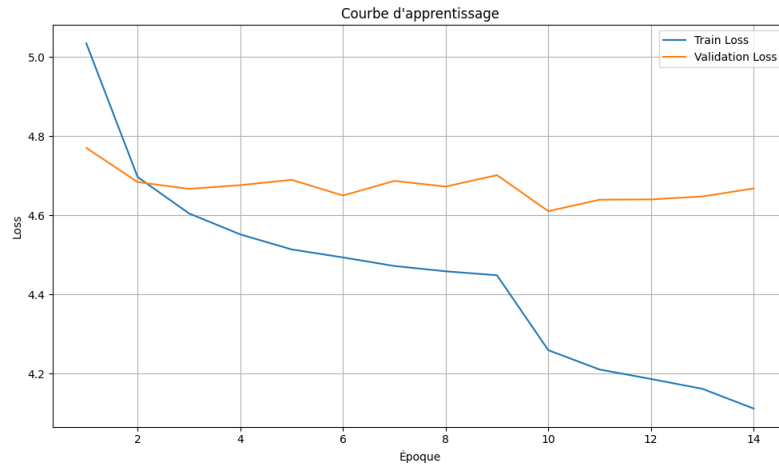


FIGURE 3 – Courbe d'apprentissage sur les données MARS montrant un surapprentissage

L'algorithme intègre plusieurs mécanismes de régularisation afin de limiter le surapprentissage et de garantir une bonne généralisation du modèle. Un arrêt précoce est mis en place : l'entraînement est automatiquement interrompu s'il n'y a pas d'amélioration des performances sur l'ensemble de validation pendant six époques consécutives. Cette stratégie permet d'éviter que le modèle continue à apprendre inutilement et commence à s'ajuster au bruit des données d'entraînement.

Par ailleurs, une validation continue est effectuée à chaque époque, permettant d'évaluer en temps réel les performances du modèle sur un jeu de données séparé, non utilisé pour l'apprentissage. Ce suivi constant est complété par un système de points de contrôle qui enregistre régulièrement les versions les plus performantes du modèle, afin de pouvoir revenir à la meilleure configuration observée. Enfin, une surveillance systématique des pertes d'entraînement et de validation est réalisée, ce qui permet d'identifier rapidement d'éventuels problèmes comme le surapprentissage ou une mauvaise convergence.

4.3.5 Paramètres d'Entraînement

Les hyperparamètres utilisés pour l'entraînement ont été choisis pour favoriser une bonne convergence du modèle. Le nombre maximal d'époques a été fixé à 50, ce qui offre une durée d'apprentissage suffisante tout en permettant d'exploiter les mécanismes d'arrêt précoce quand c'est nécessaire. La taille de lot (batch) est de 8, un choix souvent adapté aux contraintes mémoire tout en assurant une bonne stabilité des gradients.

L'optimiseur utilisé est AdamW, avec un taux d'apprentissage initial de $1e-4$, ce qui permet un apprentissage progressif sans mises à jour trop brutales. Enfin, une décroissance des poids (weight decay) de 0.01 est appliquée, correspondant à une régularisation de type L2, afin de limiter la complexité excessive du modèle en pénalisant les poids trop grands.

4.4 Architecture du Pipeline de Réglage Fin (fine tuning)

Le pipeline de fine-tuning a été conçu pour adapter efficacement notre modèle pré-entraîné aux spécificités de notre environnement de laboratoire, tout en préservant les connaissances génériques acquises sur le dataset MARS. Pour ce faire, j'ai annoté plusieurs frames de souris à partir de vidéo fournis par l'université de Valence (10000 environ) sur lequel on a appliqué plusieurs transformations. La figure 4 montre des exemples d'images annotées et transformées.

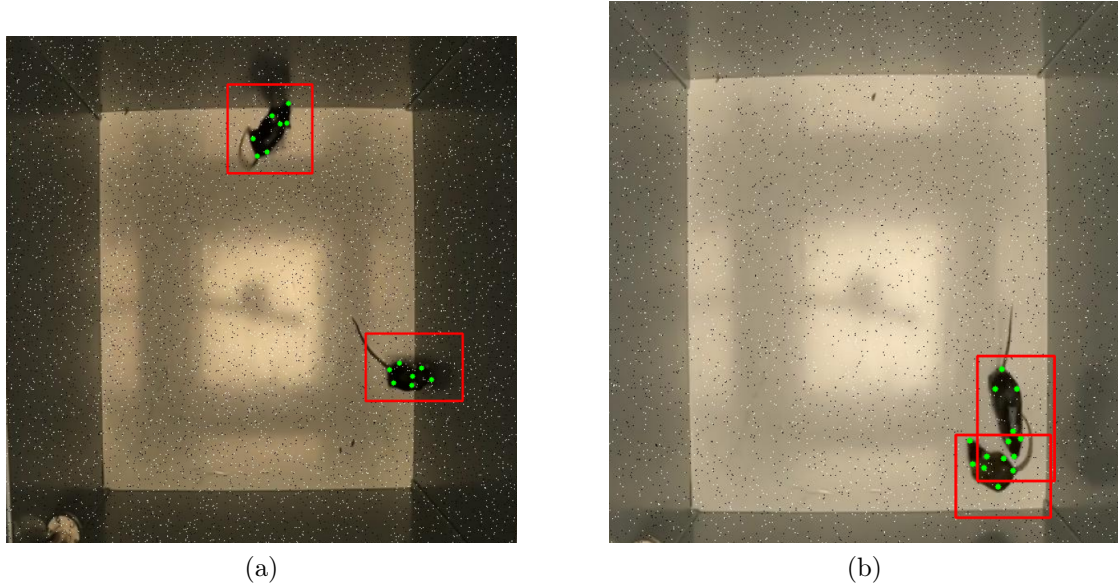


FIGURE 4 – Exemples d'images annotées et transformées

5 Résultats et Améliorations

5.1 Gains de Performance

Nous remarquons sur la figure 5 que la valeur de perte pour l'entraînement et validation baissent remarquablement, les valeurs sur le premier graphe (figure 3) convergeaient à une valeur de 4, après notre adaptation du modèle sur les vidéos du laboratoire, les valeurs convergent vers un seuil de 0.8 ce qui montre que le modèle apprend à détecter plus efficacement.

5.2 Analyse des Améliorations

L'analyse des résultats obtenus met en évidence deux types d'améliorations majeures, en termes de perte et de précision du modèle.

D'une part, on observe une réduction significative de la perte, résultat d'une meilleure adaptation du modèle aux conditions spécifiques de nos souris. Cette amélioration est rendue possible par une optimisation fine des hyperparamètres, qui permet de mieux exploiter la capacité du modèle, et par l'intégration de mécanismes de

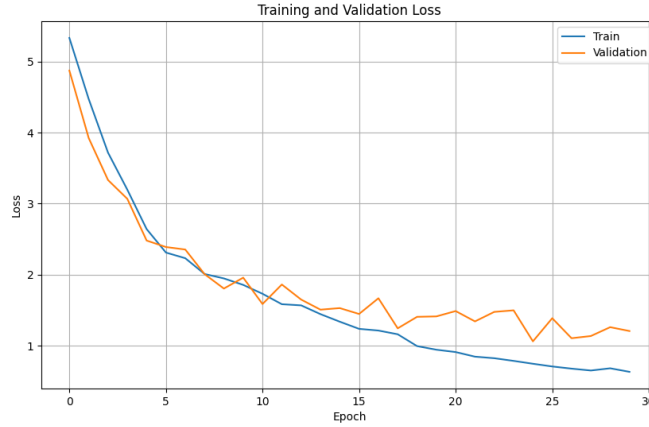


FIGURE 5 – courbe d’apprentissage

régularisation efficaces lors de la phase de fine-tuning. Ces ajustements contribuent à stabiliser l’apprentissage et à éviter le surapprentissage, tout en maintenant une bonne capacité de généralisation.

D’autre part, on note un gain net en précision, notamment dans la détection des points anatomiques sur les images. Le modèle parvient à identifier ces points avec une meilleure exactitude, ce qui se traduit par une réduction du nombre de faux positifs. En outre, le système se montre plus robuste face aux variations de pose ou de conditions d’éclairage, ce qui confirme l’efficacité des choix architecturaux et des ajustements apportés au cours de l’entraînement.

6 Pipeline de Données

6.1 Chargement et Prétraitement des Données

Le chargement efficace des données est crucial pour l’entraînement du modèle. Nous avons implémenté une classe personnalisée `MouseKeypointDataset` qui gère le chargement et les transformations des données d’une manière optimisée. (voir Annexe C).

6.2 Jeu de données

Le jeu de données MARS (Mouse Action Recognition System)¹ est une base de données publique spécialisée pour la détection et le suivi de souris de laboratoire.

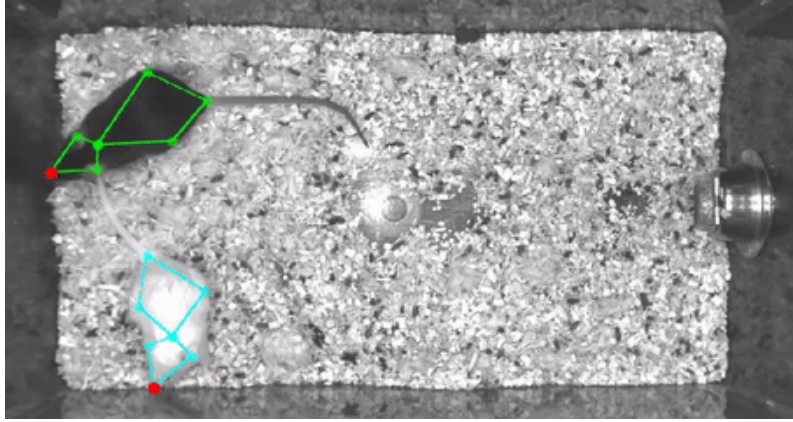


FIGURE 6 – Exemple d'image du jeu de données MARS

Le jeu de données utilisé dans ce projet se distingue par son volume de données et sa qualité. Il comprend plus de 10 000 images annotées, extraites de séquences vidéo capturant le comportement de souris en laboratoire. Toutes les annotations ont été réalisées manuellement, puis validées par des experts afin de garantir leur fiabilité et leur cohérence.

La structure des annotations est également particulièrement soignée. Chaque souris est décrite à travers sept points anatomiques clés, spécifiquement définis pour permettre une analyse fine des postures : le nez (point 0), les deux oreilles (points 1 et 2), le cou (point 3), les flancs gauche et droit (points 4 et 5), ainsi que la base de la queue (point 6). Pour chaque point, un score de visibilité est renseigné, permettant d'indiquer si le point est clairement observable sur l'image. De plus, chaque instance de souris est associée à une boîte englobante, facilitant l'entraînement des modèles de détection et de localisation.

L'algorithme de chargement (voir annexe C) a été mis en place pour assurer une gestion rigoureuse des données en amont de l'apprentissage. Il comprend des étapes de lecture, de validation et de transformation des données brutes en un format exploitable par les modèles de deep learning. Les différentes informations (images, annotations, scores de visibilité, boîtes englobantes) sont ainsi récupérées, vérifiées et converties de manière structurée.

Cet algorithme présente plusieurs propriétés essentielles qui renforcent la robustesse et la flexibilité du pipeline. Il permet notamment une gestion efficace des situations avec plusieurs souris dans une même image, en attribuant les annotations à chaque individu de manière distincte. Il intègre également des techniques simples mais efficaces d'augmentation de données, comme le retournement horizontal aléatoire, afin d'améliorer la généralisation du modèle. En outre, la conversion automatique des formats garantit la compatibilité avec les modèles utilisés, tandis que la génération dynamique des boîtes englobantes assure une mise à jour cohérente de l'ensemble des données annotées.

1. <https://neuroethology.github.io/MARS/>

Cette architecture est pensée pour être modulaire et extensible. Elle facilite le suivi comportemental précis des souris tout en rendant possible l'intégration future de nouvelles fonctionnalités. Elle est conçue pour être facilement réutilisable dans différents contextes expérimentaux, notamment ceux impliquant des variantes de comportement ou des contraintes environnementales différentes. Grâce à cette flexibilité, le système peut également évoluer vers des analyses comportementales plus complexes ou multi-espèces.

7 Implémentation du logiciel de Suivi

7.1 Architecture Logicielle

Notre système de suivi et d'analyse comportementale des souris repose sur une architecture modulaire composée de deux composants principaux : une interface graphique (GUI) développée avec PyQt5 et un processus distinct (worker) dédié au traitement des images. Cette séparation permet une exécution asynchrone du traitement lourd tout en maintenant une interface réactive. Le worker communique avec l'interface via des signaux (signals) PyQt5, permettant la mise à jour en temps réel des résultats et de la progression du traitement.

L'architecture adopte le modèle de conception Observer, où le worker émet des signaux pour notifier l'interface des changements d'état et des résultats de traitement. Cette approche garantit une séparation claire des responsabilités et facilite la maintenance du code.

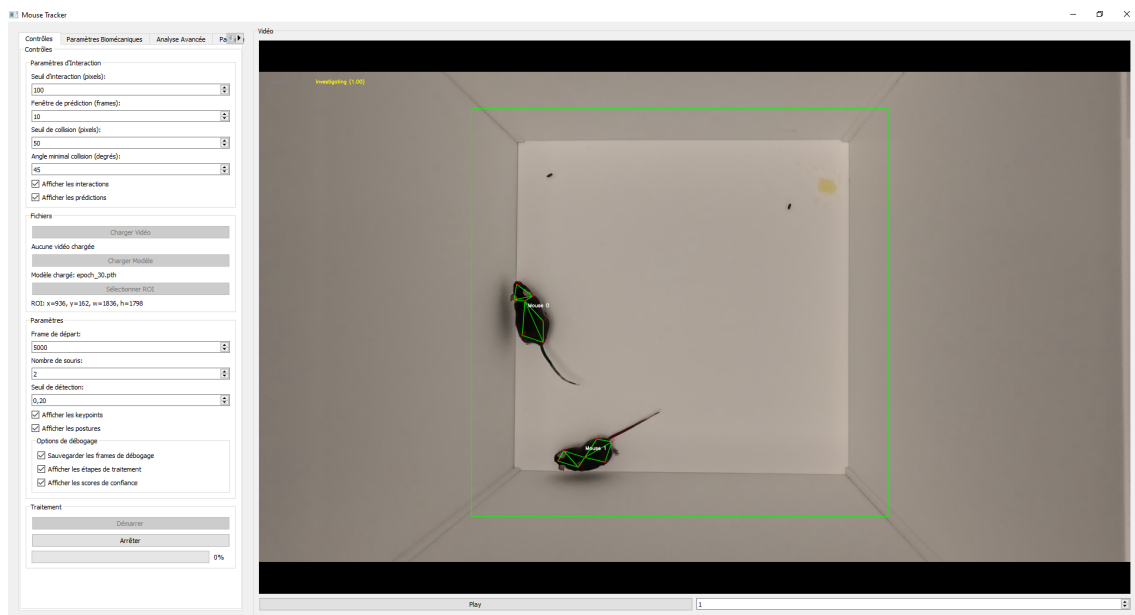


FIGURE 7 – Interface graphique

7.2 Interface Graphique (GUI)

L'interface graphique constitue l'élément central de notre logiciel de suivi, offrant aux utilisateurs un accès intuitif et complet aux fonctionnalités de suivi et d'analyse comportementale. Développée avec PyQt5, cette interface adopte une architecture modulaire organisée en plusieurs panneaux spécialisés, chacun dédié à des aspects spécifiques du traitement et de l'analyse des données.

7.2.1 Panneau de Contrôle Principal

Le panneau de contrôle principal sert de point d'entrée central pour l'utilisateur, regroupant les fonctionnalités essentielles de chargement et de configuration du système. La section de chargement des fichiers permet l'importation des vidéos expérimentales et des modèles pré-entraînés, avec une gestion automatique des formats supportés et une validation des fichiers avant traitement. L'utilisateur peut également définir une région d'intérêt (ROI) pour se concentrer sur des zones spécifiques de la vidéo, optimisant ainsi les performances de détection et réduisant les faux positifs.

Les paramètres de base offrent un contrôle granulaire sur le processus de suivis. Le paramètre de "frame" (image) de départ permet de commencer l'analyse à un moment précis de la vidéo, particulièrement utile pour éviter les périodes d'initialisation ou se concentrer sur des segments comportementaux spécifiques. Le nombre de souris à suivre, configurable, s'adapte aux différents protocoles expérimentaux et optimise les ressources de calcul en fonction du nombre d'animaux présents. Le seuil de détection, ajustable entre 0.1 et 1.0, permet de calibrer la sensibilité du système selon la qualité des images et les conditions d'éclairage.

Les options d'affichage enrichissent l'expérience utilisateur en permettant la visualisation en temps réel des keypoints détectés, des postures identifiées et des informations de débogage. Cette transparence du système facilite la validation des résultats et l'identification rapide des problèmes potentiels.

7.2.2 Panneau des Paramètres Biomécaniques

Le panneau des paramètres biomécaniques constitue l'interface avancée pour la configuration des contraintes anatomiques et physiologiques du système. La rigidité globale, paramètre principal ajustable entre 0.0 et 1.0, contrôle l'intensité des corrections biomécaniques appliquées aux détections. Une valeur élevée favorise la cohérence anatomique au détriment de la fidélité aux détections brutes, tandis qu'une valeur faible préserve les détections originales avec un risque accru d'implausibilité biomécanique.

Les seuils de confiance différenciés pour la tête, le corps et la queue reflètent la variabilité de précision de détection selon les parties anatomiques. La tête, généralement mieux détectée grâce à ses caractéristiques distinctives, bénéficie d'un seuil de confiance plus élevé (0.7), tandis que la queue, plus difficile à identifier en raison de sa mobilité et de sa taille réduite, utilise un seuil plus permissif (0.4). Le corps, avec ses caractéristiques intermédiaires, se voit attribuer un seuil de 0.5, équilibrant précision et robustesse.

Les contraintes anatomiques, notamment le seuil de symétrie, garantissent la cohérence physiologique des poses détectées. Ce paramètre vérifie la symétrie bilatérale des structures anatomiques comme les oreilles et les flancs, rejetant ou corrigeant automatiquement les détections présentant des asymétries excessives.

7.2.3 Panneau d'Analyse Avancée

Le panneau d'analyse avancée (figure 8) déploie les fonctionnalités sophistiquées d'analyse comportementale et de prédiction. L'analyse des trajectoires fournit une visualisation dynamique des déplacements des souris, avec des représentations graphiques colorées et des métriques de vitesse instantanée. Le système de prédiction des collisions utilise les filtres de Kalman pour extrapoler les trajectoires futures et identifier les risques d'interactions, avec une fenêtre de prédiction ajustable selon les besoins de l'utilisateur.

L'orientation de la tête (figure 9), paramètre crucial pour l'analyse comportementale, fait l'objet d'un suivi spécialisé. L'affichage en temps réel de l'orientation, couplé à un historique graphique, permet d'identifier les patterns comportementaux comme l'exploration, la vigilance ou l'interaction sociale. La distribution des angles, présentée sous forme d'histogramme, offre une vue statistique des préférences d'orientation de chaque souris.

Cette architecture modulaire de l'interface graphique assure une séparation claire des responsabilités tout en maintenant une cohérence visuelle et fonctionnelle. Chaque panneau peut être développé ou modifié indépendamment, facilitant les évolutions futures du système et l'adaptation à de nouveaux protocoles expérimentaux.

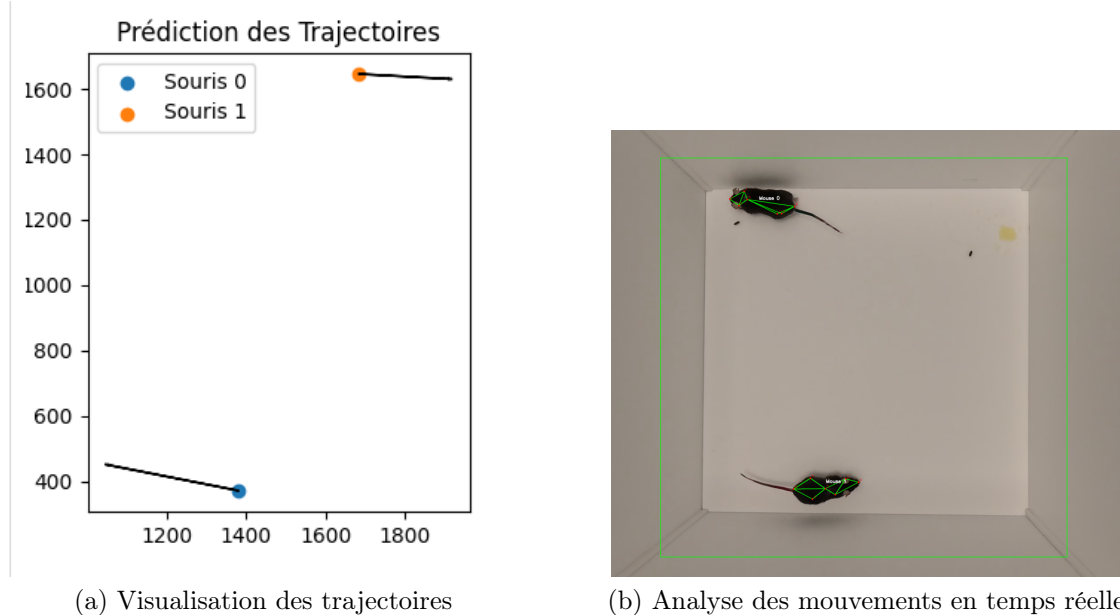


FIGURE 8 – Panneau d'analyse avancées pour visualiser la prédiction des trajectoires.

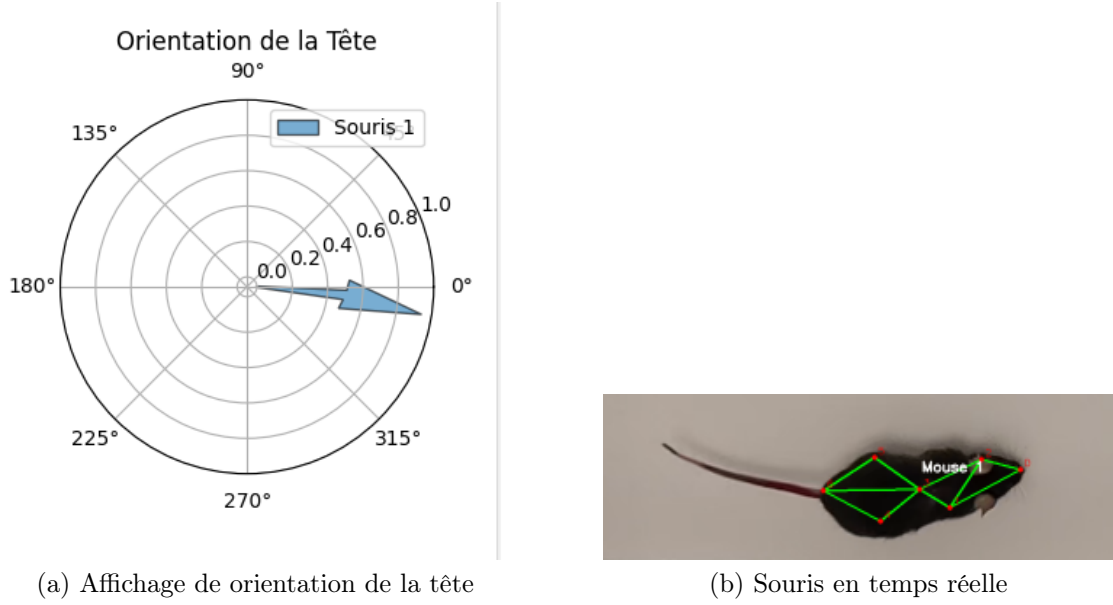


FIGURE 9 – Orientation estimée de la tête de la souris

8 Détail de la Pipeline

8.1 Modèle de détection

Notre système utilise une architecture de réseau neuronal basée sur Keypoint R-CNN, spécifiquement adaptée pour la détection des 7 points anatomiques clés chez la souris : le nez, les oreilles gauche et droite, le cou, les flancs gauche et droit, et la base de la queue. Le modèle a été modifié pour optimiser la détection de ces points spécifiques. Il est appelé :

$$\text{KeypointPredictor}(x) \quad (1)$$

où x représente les caractéristiques extraites par le backbone (structure de base) ResNet50-FPN.

8.2 Prétraitement des images

Le prétraitement des images comprend plusieurs étapes cruciales qui sont détaillées ci-dessous :

1. **Redimensionnement adaptatif** : Celui-ci est calculé de la manière suivante :

$$\text{scale_factor} = \begin{cases} 800/\text{height} & \text{si height} > \text{width} \\ 800/\text{width} & \text{sinon} \end{cases} \quad (2)$$

2. **Normalisation** : Les images sont normalisées selon les standards ImageNet² :

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}, \text{ où } \mu = [0.485, 0.456, 0.406], \sigma = [0.229, 0.224, 0.225] \quad (3)$$

2. <https://www.image-net.org/>

8.3 Post-traitement des détections

Le post-traitement est constitué de 3 étapes : l'élimination des détections redondantes, le filtrage et la gestion des cas particuliers. Ces étapes sont décrites ci-dessous.

8.3.1 Non-Maximum Suppression (NMS)

Pour éliminer les détections redondantes, nous appliquons l'algorithme de NMS. Le but de l'algorithme de NMS est donc de sélectionner le meilleur cadre de détection pour un même objet et ainsi de retirer tous les autres cadres. Nous utilisons pour faire cette sélection l'indice de Jaccard (ou IOU, Intersection Over Union). Il est calculé par la formule suivante :

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (4)$$

Où A et B sont respectivement les boites comparées. Les boîtes sont conservées si leur IoU est inférieur à un seuil de 0.3, optimisé empiriquement pour notre cas d'usage.

8.3.2 Filtrage et assignation par souris

Le processus de sélection et fusion des détections suit plusieurs étapes :

1. **Filtrage initial par score** : il est appliquée en utilisant la formule suivante :

$$\text{Détections}_{\text{valides}} = \{d_i | \text{score}(d_i) > \text{seuil}_{\text{confiance}}\} \quad (5)$$

où le seuil de confiance $\text{seuil}_{\text{confiance}}$ est par défaut égal à $= 0.5$.

2. **Sélection par souris** : Pour éviter qu'une même souris ne soit associée à plusieurs détections, nous :
 - Trions les détections par score décroissant
 - Sélectionnons les N meilleures détections (N = nombre de souris)
 - Vérifions la distance spatiale entre les détections en appliquant la règle suivante :

$$\text{Distance}(d_i, d_j) > \text{seuil}_{\text{spatial}} \quad (6)$$

3. **Fusion des détections proches** : Pour chaque paire de détections (d_i, d_j) nous appliquons la règle de décision suivante :
 - Si $\text{Distance}(d_i, d_j) < \text{seuil}_{\text{fusion}}$ et $\text{Area}(d_i \cup d_j) < k \cdot \text{Area}_{\text{mouse}}$:

$$d_{\text{fusionnée}} = \text{moyenne_pondérée}(d_i, d_j, \text{scores}_i, \text{scores}_j) \quad (7)$$

— Sinon, les souris sont considérées comme deux souris distinctes.

4. **Validation biomécanique** : Enfin, pour chaque détection finale, nous vérifions la plausibilité anatomique des points anatomiques détectés, puis nous validons les proportions et angles entre les parties du corps, enfin nous rejetons (ou corrigeons) les poses non plausibles.

8.3.3 Gestion des cas particuliers

L'algorithme de suivi intègre plusieurs mécanismes spécifiques pour gérer des situations complexes fréquemment rencontrées lors de l'analyse de séquences vidéo impliquant plusieurs souris.

Dans les cas où une **zone de détection semble regrouper deux souris** (ce qui peut être déduit par une surface anormalement grande ou un nombre excessif de points anatomiques visibles) un algorithme de clustering, tel que le K-means, est appliqué. Celui-ci permet de regrouper les points caractéristiques selon leur proximité spatiale, et ainsi de séparer les deux individus détectés à tort comme une seule entité.

Lorsque **deux souris sont proches mais doivent rester considérées comme distinctes**, la décision de les fusionner ou non repose sur deux critères principaux : la plausibilité anatomique des poses détectées et la cohérence de leur historique de mouvement. Cela se formalise par la règle suivante :

- Si $\text{Distance}(d_i, d_j) < \text{seuil}_{\text{fusion}}$ et $\text{Area}(d_i \cup d_j) < k \cdot \text{Area}_{\text{mouse}}$:

$$d_{\text{fusionnée}} = \text{moyenne_pondérée}(d_i, d_j, \text{scores}_i, \text{scores}_j) \quad (8)$$

- Sinon, les souris sont considérées comme deux souris distinctes.

Ainsi, si les postures détectées sont réalistes et compatibles avec les données temporelles précédentes, la séparation est conservée. Dans le cas contraire, une fusion des détections est envisagée pour corriger une segmentation erronée.

Enfin, pour les situations où certaines détections sont absentes ou manquantes, le système s'appuie sur l'historique des positions précédentes, en le combinant à un filtre de Kalman. Cela permet de prédire la position probable de l'individu concerné à l'instant t , selon l'équation suivante :

$$\text{position}_{\text{prédite}} = \text{Kalman}(\text{historique}_{\text{positions}}) \quad (9)$$

L'ensemble de cette approche multi-critères vise à maintenir un suivi stable et cohérent dans le temps, en évitant notamment les erreurs d'assignation multiples à une même souris, en préservant la distinction entre individus proches, en corrigeant les erreurs de fusion ou de séparation, et en assurant la continuité du suivi même en cas de détection intermittente.

Gestion des Interactions et Suivi d'Identité

8.4 Assignation Stable des IDs

Le système de suivi implémente une stratégie robuste d'assignation d'identité permettant de maintenir la cohérence des trajectoires individuelles des souris tout au long de la séquence vidéo. Cette approche repose sur l'interaction entre des identifiants persistants, une modélisation temporelle, et un algorithme d'optimisation.

8.4.1 Identifiants Globaux

Chaque souris détectée se voit attribuer un ID global fixe qui est conservé tout au long de la vidéo. Pour garantir cette stabilité temporelle, plusieurs mécanismes sont mis en œuvre :

- Un historique temporel est associé à chaque ID via une structure de type *mouse_hhistory, stockant les informations pertinentes sur une fenêtre glissante de 30 frames*.
- Chaque souris est suivie par un filtre de Kalman individuel, qui prédit sa trajectoire future à partir des observations passées, permettant de compenser les éventuelles absences de détection ou fluctuations brusques.

8.4.2 Algorithme d'Assignment

L'attribution des IDs à chaque nouvelle détection repose sur une procédure en trois étapes :

1. **Construction de la matrice de coût** : Une matrice C_{ij} est construite pour quantifier la compatibilité entre chaque détection actuelle i et les identifiants existants j . Elle est définie par la formule :

$$C_{ij} = w_h \cdot \|p_i - p_j\|_2 \quad (10)$$

où :

- p_i est la position actuelle de la détection i
- p_j est la position précédente de l'ID j
- w_h est un poids temporel ajusté dynamiquement selon la longueur de l'historique : $w_h = 0.7 + 0.3 \cdot \frac{|H_j|}{H_{max}}$ avec $|H_j|$ la taille actuelle de l'historique de l'ID j et $H_{max} = 30$ le maximum autorisé.

2. **Optimisation de l'assignation** : Une fois la matrice de coût définie, l'assignation optimale est obtenue en résolvant un problème d'optimisation combinatoire via l'algorithme hongrois (`linear_sum_assignment`). L'objectif est de minimiser le coût global :

$$\min \sum_{i,j} C_{ij} \cdot X_{ij} \quad (11)$$

sous contrainte que $X_{ij} \in \{0, 1\}$, garantissant une assignation unique entre détections et IDs.

3. **Validation et correction** : L'assignation obtenue est ensuite filtrée et ajustée à l'aide de règles de validation :
 - Un seuil maximal de coût (100 pixels) est appliqué pour exclure les appariements irréalistes.
 - Des cas particuliers, tels que la disparition temporaire ou la réapparition d'un individu, sont gérés via un mécanisme de repli, utilisant les dernières positions connues pour maintenir la continuité du suivi.
 - En cas d'échec de l'assignation, l'identité peut être temporairement suspendue jusqu'à ce que la détection devienne à nouveau fiable.

Ce dispositif permet d'assurer une traçabilité individuelle fiable, même dans des contextes complexes où les individus se croisent, disparaissent momentanément ou présentent des trajectoires non linéaires.

8.5 Gestion des Interactions et Enchevêtrements

Le système de suivi a été spécifiquement conçu pour gérer de manière fiable les situations complexes d'interactions rapprochées, d'enchevêtrements et d'occlusions fréquentes entre souris en contexte expérimental. Cette robustesse repose sur une série de modules spécialisés.

8.5.1 Détection des Rapprochements

Afin d'anticiper les risques d'enchevêtrement, le système analyse en continu les dynamiques spatiales entre individus grâce aux mécanismes suivants :

- Prédiction des trajectoires sur une fenêtre temporelle glissante, à l'aide des filtres de Kalman.
- Calcul des distances inter-souris et des angles de convergence entre trajectoires, pour détecter les mouvements de rapprochement.
- Identification précoce des risques de collision, en tenant compte des vitesses relatives des individus.

8.5.2 Séparation des Détections Fusionnées

Lorsque deux souris se retrouvent spatialement proches au point que leurs détections se superposent, une procédure de séparation est déclenchée.

1. **Détection de fusion** : Une alerte est levée si la surface d'une boîte englobante dépasse un seuil proportionnel à la taille moyenne d'une souris :

$$A_{box} > k \cdot (W_{mouse} \cdot H_{mouse}) \quad (12)$$

avec $k = 0.8$, et W_{mouse}, H_{mouse} représentant les dimensions moyennes d'un individu.

2. **Clustering des keypoints** : Une fois la fusion détectée, les points caractéristiques visibles sont séparés via un algorithme de clustering K-means, puis chaque cluster est soumis à une validation biomécanique. Des squelettes distincts sont alors reconstruits à partir des points séparés.
3. **Validation des poses reconstruites** : Cette étape consiste à contrôler la plausibilité anatomique de chaque pose, appliquer des corrections biomécaniques si nécessaire et enfin filtrer automatiquement les reconstructions non conformes.

8.5.3 Gestion des Occlusions

Les situations d'occlusion partielle ou totale de certaines souris sont traitées à l'aide d'approximations temporelles en réalisant les actions suivantes :

- L'interpolation linéaire des points caractéristiques manquants sur un maximum de 10 images consécutives.
- La prédiction des positions à l'aide des filtres de Kalman, en se basant sur l'historique de mouvement.
- Maintien de l'identité via un ancrage sur la dernière position connue et sa prédiction dynamique.

8.5.4 Métriques de Qualité

Pour garantir un suivi précis et évaluable, le système calcule plusieurs métriques en continu :

- Taux de détection global des souris présentes.
- Stabilité des IDs, mesurée par le nombre de réassignations d'identifiants.
- Qualité des poses, évaluée via un score de plausibilité biomécanique.
- Taux de réussite de séparation dans les cas de rapprochement ou d'enchevêtrement.

Filtre de Kalman pour le Suivi

Le filtre de Kalman s'intègre de manière centrale dans le système de suivi des souris, en apportant plusieurs fonctionnalités clés qui améliorent la robustesse et la précision du tracking. Il permet tout d'abord de prédire les positions futures des souris, ce qui est essentiel pour anticiper d'éventuelles collisions ou chevauchements entre trajectoires.

Par ailleurs, le filtre joue un rôle crucial dans la gestion des occlusions temporaires : lorsqu'une souris est partiellement ou totalement masquée, il assure la continuité du suivi en s'appuyant sur la dynamique du mouvement précédemment observé.

Il permet également de filtrer le bruit inhérent aux détections image par image, en lissant les trajectoires de manière cohérente sans altérer la réactivité du système. Enfin, le filtre fournit une estimation instantanée des vitesses des souris, ce qui constitue une information utile à la fois pour la dynamique du suivi et pour l'analyse comportementale.

Les détails sur l'implémentation du filtre de Kalman sont donnés en annexe D.

Corrections biomécaniques

Le système intègre un mécanisme de correction des poses visant à garantir leur plausibilité biomécanique, en s'appuyant sur des contraintes anatomiques propres à la morphologie des souris.

Dans un premier temps, une validation anatomique est effectuée. Celle-ci repose sur deux critères principaux : la symétrie gauche/droite, qui vérifie l'équilibre des positions des membres ou organes bilatéraux par rapport à l'axe du corps, et la cohérence angulaire entre certaines parties du corps, notamment entre le nez, le cou et la queue. Ces angles doivent rester dans des plages compatibles avec la posture naturelle des animaux.

Ensuite, le système procède à une correction des poses lorsque des incohérences sont détectées. Deux niveaux d'ajustement sont mis en œuvre :

- Une correction globale, qui consiste à réaligner l'ensemble de la pose vers une configuration plausible, estimée à l'aide d'un modèle rigide adaptatif, prenant en compte les contraintes anatomiques globales de l'animal.
- Une correction locale, qui ajuste les distances entre certaines paires de points du squelette (par exemple entre les oreilles et le cou, ou entre le cou et la base

de la queue) afin de les ramener dans des plages de valeurs physiologiquement plausibles.

Ce processus contribue à renforcer la cohérence spatio-temporelle des trajectoires suivies et assure la validité biomécanique des postures estimées, en évitant des configurations anatomiquement impossibles ou hautement improbables.

Les détails sur l'implémentation des corrections biomécaniques sont donnés en annexe D.

9 Pipeline de Traitement

Le système de tracking des souris suit un pipeline de traitement en plusieurs étapes, chacune conçue pour améliorer la précision et la fiabilité des détections. Les sections suivantes décrivent brièvement ces étapes.

9.1 Détection Brute

La première étape consiste en une détection brute des points caractéristiques à l'aide du modèle de deep learning personnalisé. Le modèle utilise une architecture R-CNN modifiée pour détecter la boîte englobante contenant la souris et les 7 points clés sur chaque souris :

- Nez
- Oreilles (gauche et droite)
- Base du cou
- Flancs (gauche et droite)
- Base de la queue

La figure 10 montre un exemple de sortie brute.

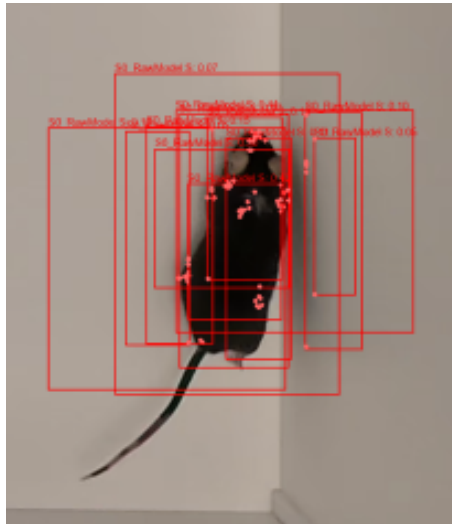


FIGURE 10 – Sortie brute du modèle de détection

9.2 Filtrage NMS et Seuillage

Après la détection brute, nous appliquons, l'algorithme de NMS (Non-Maximum Suppression) pour éliminer les détections redondantes, puis un seuillage des scores de confiance pour éliminer les fausses détections.

La figure 11 montre le résultat obtenu après élimination des boites redondantes et l'opération de filtrage.



FIGURE 11 – Résultat après application de NMS et du seuillage

9.3 Sélection des N "Meilleures" Souris

Le système sélectionne ensuite les N meilleures détections de souris en fonction des Scores de confiance globaux, du nombre de points caractéristiques visibles en imposant une détection (une boîte englobante) par souris.

La figure 12 montre le résultat après sélection de la "meilleure" détection.

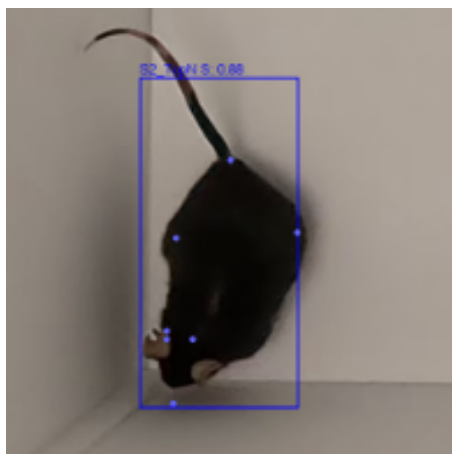


FIGURE 12 – Sélection de la meilleure détection.

9.4 Correction Anatomique

L'application des contraintes biomécaniques permet ensuite de corriger les détections en imposant que les points anatomiques respectent les règles suivantes :

- Symétrie des oreilles et des flancs
- Contraintes d'angle pour le corps
- Correction de la position de la queue

La figure 13 montre le résultat avant et après l'application de la correction anatomique des points caractéristiques détectés.

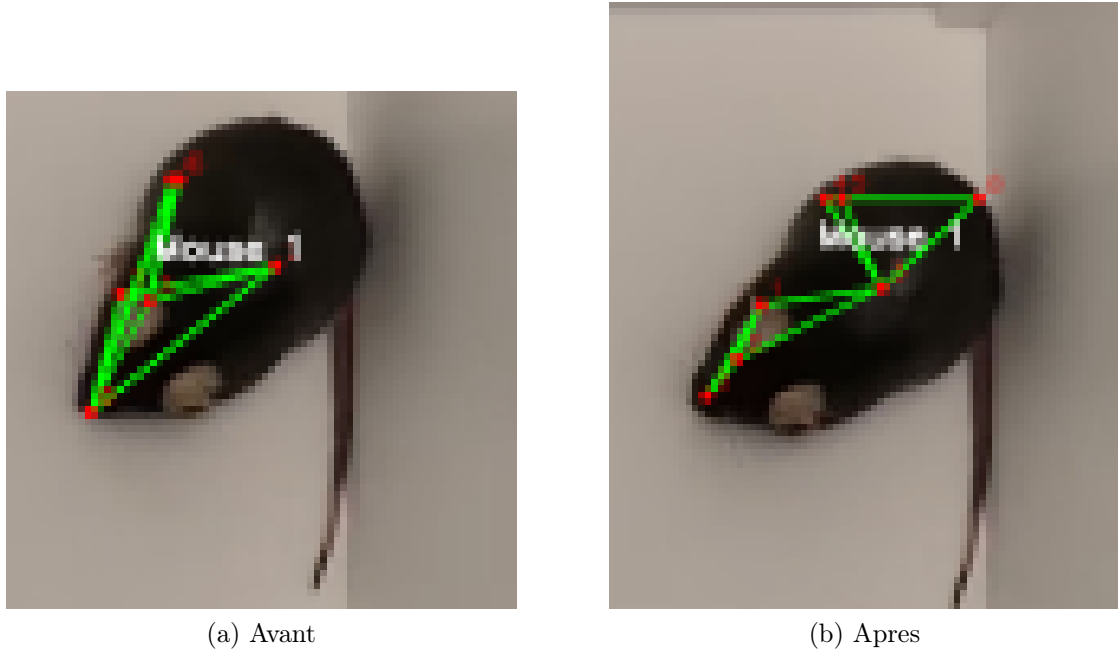
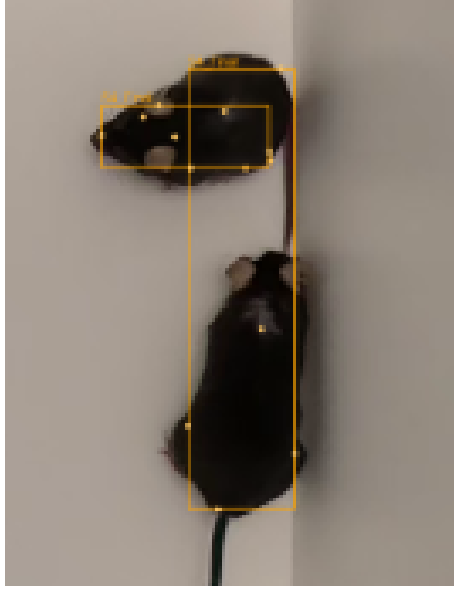


FIGURE 13 – Exemple de détection des points caractéristiques avant et après la correction anatomique.

9.5 Gestion de la proximité des souris

Lorsque les souris sont proches, la détection génère une boîte englobant deux souris, soit partiellement, soit totalement. La prise en compte de la position des points anatomiques détectés permet de corriger ces erreurs de détections.

La figure 14 montre un exemple de détection erronée où l'identification des souris a pu être corrigé à partir des points anatomique détectés.



(a) Avant



(b) Apres

FIGURE 14 – Exemple de correction en considérant les points anatomiques lorsque les souris sont proches.

9.6 Fichier de sortie

L'interface permet également l'export des résultats sous différents formats (XML, CSV, vidéo annotée) pour des analyses ultérieures.

La figure 15 montre un exemple de fichier XML généré par le système de suivi. La structure du fichier XML décrit pour chaque image (frame) de la vidéo, les coordonnées des 7 points anatomiques de chaque souris détectée.

Métriques

Le système est évalué selon plusieurs métriques :

1. Précision de détection :

$$PCK@_{\alpha} = \frac{\text{points correctement détectés}}{\text{total des points}} \text{ où } \text{dist} < \alpha \cdot \text{scale} \quad (13)$$

2. Stabilité temporelle :

$$\text{stabilité} = \frac{1}{N} \sum_{t=1}^{N-1} \|\text{pose}_t - \text{pose}_{t+1}\|_2 \quad (14)$$

```

<VideoAnnotations>
  <Frame index="1500">
    <Mouse id="0">
      <Keypoint id="0" x="260.07266" y="1671.3558"/>
      <Keypoint id="1" x="253.83754" y="1593.3776"/>
      <Keypoint id="2" x="253.83754" y="1602.735"/>
      <Keypoint id="3" x="281.89557" y="1602.735"/>
      <Keypoint id="4" x="394.12766" y="1487.3269"/>
      <Keypoint id="5" x="263.19022" y="1493.5652"/>
      <Keypoint id="6" x="322.42383" y="1409.3486"/>
    </Mouse>
    <Mouse id="1">
      <Keypoint id="0" x="1528.5396" y="327.62848"/>
      <Keypoint id="1" x="1528.5396" y="343.18976"/>
      <Keypoint id="2" x="1550.5391" y="293.39368"/>
      <Keypoint id="3" x="1566.2529" y="324.51624"/>
      <Keypoint id="4" x="1597.6809" y="402.3226"/>
      <Keypoint id="5" x="1641.6799" y="420.99612"/>
      <Keypoint id="6" x="1676.2506" y="433.44513"/>
    </Mouse>
  </Frame>

```

FIGURE 15 – Exemple de fichier XML contenant les données de suivi des souris (points caractéristique, numéro de frame, etc.).

Futurs travaux

Afin d'améliorer le système que nous avons développé, il serait intéressant d'explorer les architectures temporelles.

En complément des approches basées sur la détection image par image, nous avons envisagé le développement d'un modèle combinant des réseaux de neurones convolutifs (CNN) et des réseaux de neurones récurrents (LSTM) pour exploiter la dimension temporelle des vidéos. L'objectif serait de capturer les dynamiques comportementales à plus long terme et d'améliorer la robustesse du suivi.

Architecture possible Le modèle CNN-LSTM envisagé comprendrait :

- des **Couches de type CNN** pour l'extraction de caractéristiques spatiales dans chaque frame
- des **Couches LSTM** (Long Short Term Memory) pour modéliser les dépendances temporelles sur des séquences de plusieurs frames (images).

Avantages potentiels

- Meilleure robustesse aux occlusions temporaires
- Prise en compte de la dynamique du mouvement
- Prédiction plus stable des trajectoires

Difficultés pour la mise en oeuvre Nous n'avons pas pu mettre en place cette approche par manque de temps mais également pour plusieurs autres raisons :

- **Besoin en données annotées** : Contrairement à l'approche par détection image par image, un modèle séquentiel nécessite des annotations complètes sur des séquences vidéo continues

- **Complexité d’annotation** : L’annotation manuelle de séquences complètes est extrêmement chronophage (estimée à 10 fois plus longue que l’annotation d’images isolées)
- **Charge computationnelle** : L’entraînement de modèles CNN-LSTM requiert des données significativement plus importantes
- **Alignement temporel** : Il est difficile de synchroniser précisément les annotations entre frames successives

Conclusion Face à ces contraintes pratiques et en l’absence de séquences pré-annotées suffisantes, nous avons privilégié l’approche par détection statique enrichie de mécanismes de suivi temporel. Cette solution offre un compromis satisfaisant entre performance et faisabilité pratique dans le cadre temporel limité du stage. Mais cette stratégie serait intéressante à étudier.

Compétences acquises pendant le stage

Compétences Techniques

Au cours de ce stage, j’ai développé de solides compétences en vision par ordinateur. J’ai appris à détecter et suivre des objets en temps réel, notamment en implémentant des algorithmes de détection de points-clés. J’ai aussi beaucoup travaillé avec OpenCV, ce qui m’a permis de manipuler des images et des séquences vidéo de manière efficace, tout en appliquant des filtres et des techniques de post-traitement pour améliorer la qualité des données exploitées.

En intelligence artificielle, j’ai manipulé des modèles de deep learning pour la détection de postures animales. J’ai également mis en œuvre un filtre de Kalman afin de suivre les objets de manière plus robuste, ce qui m’a amené à optimiser les paramètres des modèles de tracking. J’ai aussi découvert comment ces outils pouvaient être appliqués à l’analyse comportementale, en extrayant des informations utiles à partir de données brutes.

Sur le plan du développement logiciel, j’ai approfondi ma maîtrise du langage Python, en développant des outils complets, dont des interfaces graphiques à l’aide de la bibliothèque PyQt5. J’ai également utilisé Git pour la gestion de versions, ce qui m’a permis de travailler de manière organisée et collaborative.

Enfin, j’ai acquis des compétences en analyse de données, notamment dans le traitement de données spatio-temporelles. J’ai utilisé Matplotlib pour visualiser les résultats et j’ai mené des analyses statistiques sur le comportement des souris. J’ai aussi appris à générer des rapports et à extraire des métriques pour évaluer les performances du système.

Compétences Transversales

Ce stage m’a également permis de développer plusieurs compétences transversales. En gestion de projet, j’ai appris à planifier et organiser les étapes du développement, à documenter mon travail de manière rigoureuse, et à prioriser mes tâches selon les besoins du projet, en suivant une méthodologie de travail agile et itérative.

Sur le plan de la communication, j'ai régulièrement présenté mes avancées techniques à l'équipe, rédigé des documents scientifiques et échangé avec les chercheurs pour adapter mon travail aux objectifs de l'équipe. Ces échanges ont été essentiels pour faire évoluer le projet de manière cohérente.

J'ai aussi renforcé ma capacité à résoudre des problèmes. Face aux obstacles techniques, j'ai appris à analyser les causes, à déboguer des systèmes complexes et à rechercher des solutions optimales. J'ai fait preuve de curiosité et de rigueur dans la veille technologique pour rester à jour sur les outils et approches disponibles.

Enfin, ce stage m'a permis de gagner en autonomie. J'ai souvent dû prendre des décisions techniques de manière indépendante, apprendre par moi-même de nouvelles technologies, et proposer des améliorations pour rendre le système plus efficace. J'ai également appris à mieux gérer mon temps et les ressources à ma disposition pour répondre aux objectifs du stage.

Conclusion

Au cours de ce stage, nous avons développé une solution complète et innovante pour la détection et le suivi de souris en laboratoire, marquant une avancée significative dans le domaine de l'analyse comportementale automatisée. Le système combine avec succès les dernières avancées en matière de vision par ordinateur et d'intelligence artificielle, tout en intégrant des contraintes biologiques essentielles pour garantir la pertinence des résultats.

Les principales réalisations comprennent :

- L'implémentation d'un système de détection multi-souris robuste, capable de gérer les situations complexes d'occlusion et d'interaction
- Le développement d'un module de correction anatomique sophistiqué, assurant la cohérence biologique des détections
- La création d'une interface utilisateur intuitive facilitant l'analyse des données comportementales
- L'optimisation des performances permettant une analyse en temps réel
- La mise en place d'un système complet de validation et de tests assurant la fiabilité des résultats

Les perspectives d'amélioration sont nombreuses et prometteuses :

- L'extension du système à d'autres espèces animales
- L'intégration de nouveaux modules d'analyse comportementale
- L'amélioration continue des performances via l'optimisation des algorithmes
- Le développement de fonctionnalités d'analyse statistique avancées

Ce projet a non seulement permis d'atteindre les objectifs techniques initiaux, mais a également contribué à l'avancement de la recherche en éthologie computationnelle. Les compétences acquises, tant techniques que transversales, constituent un socle solide pour de futurs développements dans le domaine de l'analyse comportementale assistée par ordinateur.

Annexes

A Bibliographie

- Grewal, M., Andrews, A. (2001). *Kalman Filtering : Theory and Practice Using MATLAB* (2nd ed.). John Wiley Sons.
- Welch, G., Bishop, G. (2006). *An Introduction to the Kalman Filter*. University of North Carolina at Chapel Hill.
- Mudimela, S., Vishwanath, N. K., Pillai, A., Morales, R., Marrelli, S. P., Barichello, T., Giridharan, V. V. (2022). Clinical significance and potential role of trimethylamine N-oxide in neurological and neuropsychiatric disorders. *Drug Discovery Today*, 27(11), 103334.
- arXiv :1504.08083
- Albinali, Hussah Abdulraheem, Fatimah. (2021). Faster R-CNN for Detecting Regions in Human-Annotated Micrograph Images. 1-6. 10.1109/WiD-STaif52235.2021.9430211.
- Yu, Yanjie Li, Qiang Zhang, Houyi. (2023). Unscented Trainable Kalman Filter Based on Deep Learning Method Considering Incomplete Information. IEEE Access. PP. 1-1. 10.1109/ACCESS.2023.3277616.
- Lin Feature Pyramid Networks CVPR 2017 paper

B Algorithme d'entraînement

Algorithm 1 Boucle d'entraînement (Trainer)

Require: Chargeurs de données, modèle, optimiseur, device

Ensure: Modèle entraîné et courbes de perte

```
1: Historique  $\leftarrow \{\}$ 
2:  $best\_loss \leftarrow \infty$ ,  $counter \leftarrow 0$ 
3: for  $epoch = 1$  à  $50$  do
4:    $total\_loss \leftarrow 0.0$ 
5:   for  $batch$  dans  $train\_loader$  do ▷ Phase d'entraînement
6:      $images, targets \leftarrow transferer\_vers\_device(batch)$ 
7:      $loss\_dict \leftarrow model(images, targets)$ 
8:      $loss \leftarrow \sum loss\_dict.values()$ 
9:      $loss.backward()$ 
10:     $optimizer.step()$ 
11:     $total\_loss \leftarrow total\_loss + loss.item()$ 
12:   end for
13:    $train\_loss \leftarrow total\_loss / len(train\_loader)$ 
14:    $val\_loss \leftarrow 0.0$  ▷ Phase de validation
15:   for  $batch$  dans  $val\_loader$  do
16:     Calculer  $loss$  similaire sans rétropropagation
17:      $val\_loss \leftarrow val\_loss + loss.item()$ 
18:   end for
19:    $val\_loss \leftarrow val\_loss / len(val\_loader)$ 
20:   if  $val\_loss < best\_loss$  then ▷ Early stopping
21:      $best\_loss \leftarrow val\_loss$ 
22:      $counter \leftarrow 0$ 
23:   else
24:      $counter \leftarrow counter + 1$ 
25:   end if
26:   if  $counter \geq 6$  then
27:     break ▷ Arrêt anticipé
28:   end if
29:   if  $epoch \bmod 2 == 0$  then ▷ Sauvegarde checkpoint
30:     Sauvegarder modèle, optimiseur, historique
31:   end if
32: end for
33: Tracer courbes de perte
```

C Chargement des données

Algorithm 2 Procédure de chargement des données (MouseKeypointDataset)

Require: Chemin du JSON d'annotations, répertoire des images

Ensure: Tuple (image, target) avec transformations appliquées

```
1: Charger les annotations JSON
2: for chaque échantillon  $i$  dans le dataset do
3:   Charger l'image  $img_i$  depuis le répertoire
4:   Convertir  $img_i$  en BGR
5:   Initialiser listes  $boxes$ ,  $keypoints$ ,  $visibilities$ 
6:   for chaque couleur de souris (noir/blanc) do
7:     Extraire coordonnées  $(x_j, y_j)$  et visibilité  $v_j$  pour  $j = 1..7$ 
8:      $kps \leftarrow [[x_1, y_1, v_1], \dots, [x_7, y_7, v_7]]$ 
9:      $box \leftarrow [\min(x), \min(y), \max(x), \max(y)]$ 
10:    Ajouter  $box$  à  $boxes$ ,  $kps$  à  $keypoints$ ,  $v$  à  $visibilities$ 
11:   end for
12:   Convertir en tenseurs PyTorch
13:   if  $random() > 0.5$  then ▷ Flip horizontal aléatoire
14:      $img_i \leftarrow flip\_horizontal(img_i)$ 
15:      $width \leftarrow width(img_i)$ 
16:      $boxes[:, [0, 2]] \leftarrow width - boxes[:, [2, 0]]$ 
17:      $keypoints[:, :, 0] \leftarrow width - keypoints[:, :, 0]$ 
18:   end if
19:    $image\_tensor \leftarrow ToTensor()(img_i)$ 
20:   Générer  $target$  dictionnaire
21: end for
```

D Filtre de Kalman pour le Suivi

Implémentation du Filtre de Kalman

Le système utilise un filtre de Kalman pour le suivi temporel des souris. Notre implémentation est spécifiquement adaptée au suivi de points anatomiques.

Modèle d'État

Le filtre utilise un modèle d'état à 4 dimensions pour chaque souris :

$$\mathbf{x} = [x, y, v_x, v_y]^T \quad (15)$$

où :

- (x, y) : Position du centroïde
- (v_x, v_y) : Vitesses dans les directions x et y

Équations de Prédiction

La phase de prédiction utilise les équations suivantes :

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}\hat{\mathbf{x}}_{k-1|k-1} \quad (16)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^T + \mathbf{Q} \quad (17)$$

où :

- $\mathbf{F}$: Matrice de transition d'état
- $\mathbf{P}$: Matrice de covariance
- $\mathbf{Q}$: Matrice de bruit de processus

Équations de Mise à Jour

La phase de mise à jour intègre les nouvelles mesures :

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}^T(\mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^T + \mathbf{R})^{-1} \quad (18)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k(\mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_{k|k-1}) \quad (19)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k\mathbf{H})\mathbf{P}_{k|k-1} \quad (20)$$

où :

- $\mathbf{K}_k$: Gain de Kalman
- $\mathbf{z}_k$: Mesure actuelle
- $\mathbf{H}$: Matrice d'observation
- $\mathbf{R}$: Matrice de bruit de mesure

Efficacité du Filtre de Kalman dans le Suivi de Souris

Adéquation avec la Nature du Mouvement

Le filtre de Kalman est particulièrement efficace dans notre contexte pour plusieurs raisons :

- **Modèle de Mouvement Adapté**
 - Le modèle à vitesse constante correspond bien au déplacement naturel des souris
 - Les changements de vitesse sont progressifs, ce qui correspond au modèle de bruit gaussien
 - La matrice de transition capture efficacement l'inertie du mouvement
- **Gestion de l'Incertitude**
 - Le filtre équilibre naturellement les prédictions et les mesures via le gain de Kalman.
 - Les matrices de bruit Q et R sont calibrées pour notre cas d'usage.

Optimisation des Matrices de Bruit Q et R

Importance des Matrices Q et R

Les matrices de bruit Q (bruit de processus) et R (bruit de mesure) sont cruciales pour la performance du filtre de Kalman :

- **Matrice Q** : Représente l'incertitude dans le modèle de mouvement
- **Matrice R** : Représente l'incertitude dans les mesures

Méthode d'Optimisation

Nous avons utilisé une approche systématique pour déterminer les valeurs optimales :

1. Analyse des Données

- Calcul de la variance des mesures sur des séquences de test
- Analyse des changements de vitesse typiques des souris
- Évaluation de la précision du modèle

2. Estimation Initiale

$$R = \begin{bmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_y^2 \end{bmatrix} \quad (21)$$

où σ_x^2 et σ_y^2 sont estimés à partir des erreurs de détection .

3. Calibration de Q

$$Q = \begin{bmatrix} \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} & 0 \\ 0 & \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & 0 & \Delta t^2 & 0 \\ 0 & \frac{\Delta t^3}{2} & 0 & \Delta t^2 \end{bmatrix} \cdot q \quad (22)$$

où q est un facteur d'échelle et Δt est l'intervalle entre frames.

Procédure d'Ajustement

Algorithm 3 Optimisation des matrices Q et R

- 1: Initialiser Q et R avec des valeurs théoriques
 - 2: **for** chaque séquence de test **do**
 - 3: Calculer l'erreur de tracking E_{track}
 - 4: Calculer la smoothness de la trajectoire S_{traj}
 - 5: Ajuster Q et R pour minimiser $\alpha E_{track} + (1 - \alpha) S_{traj}$
 - 6: **end for**
-

Critères d'Optimisation

Les valeurs optimales ont été déterminées en minimisant :

$$J(Q, R) = \alpha \sum_{t=1}^T \|x_t - \hat{x}_t\|^2 + (1 - \alpha) \sum_{t=2}^T \|\Delta v_t\|^2 \quad (23)$$

où :

- α : Compromis entre précision et lissage (0.7 dans notre cas)
- x_t : Position réelle
- $\hat{x}_t$: Position estimée
- Δv_t : Changement de vitesse

Valeurs Optimales Obtenues

Après optimisation, nous avons obtenu :

- **Pour Q :**
 - $q = 0.1$ pour le mouvement normal
 - $q = 0.3$ pour les mouvements rapides
 - $q = 0.05$ pour les phases de repos
- **Pour R :**
 - $\sigma_x = \sigma_y = 1.0$ pour les détections de haute confiance
 - $\sigma_x = \sigma_y = 2.0$ pour les détections de faible confiance

Adaptation Dynamique

Le système ajuste dynamiquement Q et R en fonction de :

- **Score de Confiance CNN**

$$R_{adaptif} = R_{base} \cdot (1 + \beta(1 - \text{confidence})) \quad (24)$$

- **Vitesse de Mouvement**

$$Q_{adaptif} = Q_{base} \cdot (1 + \gamma\|\mathbf{v}\|) \quad (25)$$

où $\beta = 2.0$ et $\gamma = 0.5$ sont des facteurs d'échelle.

Cette approche adaptative permet au filtre de :

- S'adapter aux changements de comportement
- Gérer les variations de qualité des détections
- Maintenir un équilibre optimal entre lissage et réactivité

Gestion des Cas Spéciaux

Le système gère plusieurs cas particuliers :

- **Détections manquantes :**

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} \text{ si aucune mesure disponible} \quad (26)$$

- **Réinitialisation :** Le filtre est réinitialisé si :
 - La différence entre prédiction et mesure dépasse un seuil
 - Aucune mesure n'est disponible pendant plus de 10 frames
- **Vitesses aberrantes :** Les vitesses sont limitées à des valeurs physiologiquement plausibles :

$$\|\mathbf{v}\| \leq v_{max} \text{ où } v_{max} = 50 \text{ pixels/frame} \quad (27)$$

Intégration avec le Système de Tracking

Le filtre de Kalman est utilisé pour :

- Prédire les positions futures pour la détection de collisions
- Maintenir le suivi pendant les occlusions temporaires
- Filtrer le bruit dans les trajectoires détectées
- Estimer les vitesses instantanées des souris

Corrections biomécaniques

Validation anatomique

Le système implémente une validation anatomique rigoureuse basée sur plusieurs contraintes biomécaniques :

1. Symétrie corporelle :

$$|\text{dist}(\text{oreille_gauche}, \text{cou}) - \text{dist}(\text{oreille_droite}, \text{cou})| < \epsilon_{\text{symétrie}} \quad (28)$$

2. Contraintes angulaires :

$$\angle(\text{nez}, \text{cou}, \text{queue}) \in [\theta_{\min}, \theta_{\max}] \quad (29)$$

où $\theta_{\min}$ et $\theta_{\max}$ sont déterminés empiriquement à partir de l'analyse de poses naturelles de souris.

Correction des poses

La correction des poses s'effectue en deux étapes :

1. **Correction globale** : Application d'un modèle biomécanique avec une rigidité adaptative :

$$p_{\text{corrigé}} = p_{\text{original}} + \alpha \cdot \text{stiffness} \cdot (p_{\text{prédit}} - p_{\text{original}}) \quad (30)$$

où α est un facteur d'amortissement et stiffness varie selon la pose détectée.

2. **Correction locale** : Ajustement fin des positions relatives :

$$\text{dist}(p_i, p_j) \in [\text{dist}_{\min}(i, j), \text{dist}_{\max}(i, j)] \quad (31)$$

Paramètres Biomécaniques

Les paramètres biomécaniques sont cruciaux pour la validation et la correction des détections :

Seuils de Confiance

- Tête : 0.7
- Corps : 0.5
- Queue : 0.4

Contraintes Anatomiques

- Seuil de symétrie : 15 degrés
- Seuil d'angle du corps : 30 degrés
- Rigidité biomécanique : 0.2

$$p_{\text{corrigé}} = p_{\text{original}} + 0.2 \cdot (p_{\text{prédit}} - p_{\text{original}}) \quad (32)$$